Day 24 : Function in javascript

Part 1: The Core Concept - What is a Function?

First Thought: A function is a reusable "code machine" or a "recipe."

Imagine you have a task you need to do over and over, like making a sandwich. Instead of re-writing the instructions (get bread, get cheese, etc.) every single time, you write the recipe down once and give it a name, like "Make Turkey Sandwich."

- The **recipe** is the **function**.
- The **ingredients** you give it (like "turkey" or "ham") are the **parameters**.
- The finished sandwich is the **return value**.

Why do we need them? The DRY Principle: **D**on't **R**epeat **Y**ourself. Functions let you write a piece of logic once and reuse it hundreds of times.

A. The Three Core Parts

1. **Declaration (Writing the Recipe):** This is where you define the function and give it a name.
2. **Calling (Using the Recipe):** This is where you execute the function.
3. **Parameters & Return Value (Ingredients & Finished Dish):** How you pass data *into* a function and get a result *out of* it.

B. Basic Syntax

```
// 1. DECLARATION: Writing the recipe
//    - `function` is the keyword.
//    - `greet` is the name of our function.
//    - `(name)` is the parameter, a placeholder for the input.
function greet(name) {
  // The code inside the {} is the "body" of the function.
  const message = `Hello, ${name}!`;
  return message; // 3. RETURN VALUE: Giving back the result.
```

```
}

// 2. CALLING: Using the recipe with a specific ingredient ("Alice")
const greetingForAlice = greet("Alice");
console.log(greetingForAlice); // Outputs: "Hello, Alice!"

const greetingForBob = greet("Bob");
console.log(greetingForBob);   // Outputs: "Hello, Bob!"
```

- **Parameter:** The variable name inside the function's parentheses (`name`). It's a placeholder.
- **Argument:** The actual value you pass into the function when you call it (`"Alice"`).
- **Return Value:** The value a function sends back using the `return` keyword. If a function has no `return` statement, it automatically returns `undefined`.

Part 2: Different Ways to Create Functions

There are three main ways to create functions in JavaScript, and their differences are important.

A. Function Declaration

This is the classic way we saw above.

```
function add(a, b) {
  return a + b;
}
```

Key Feature: Hoisting. Function declarations are "hoisted," meaning the JavaScript engine "lifts" them to the top of their scope before the code is executed. This means you can call a function *before* you define it in the code.

```
console.log(add(2, 3)); // 5 (This works!)

function add(a, b) {
```

```
  return a + b;
}
```

B. Function Expression

Here, you create an anonymous (unnamed) function and assign it to a variable.

```
const subtract = function(a, b) {
  return a - b;
};
```

Key Feature: Not Hoisted. A function expression is not hoisted. It behaves just like a variable. You cannot call it before it is defined.

```
// subtract(10, 5); // This would throw a ReferenceError!

const subtract = function(a, b) {
  return a - b;
};

console.log(subtract(10, 5)); // 5 (This works)
```

C. Arrow Functions (ES6 - The Modern Way)

First Thought: "A shorter, cleaner syntax for writing functions."

Arrow functions are a more concise way to write function expressions. They are extremely popular in modern JavaScript.

```
// A function expression
const multiply = function(a, b) {
  return a * b;
}

// The same function as an arrow function
const multiplyArrow = (a, b) => {
```

```
  return a * b;
}
```

Syntax Rules for Arrow Functions:

1. **Implicit Return:** If the function body is just a single expression, you can remove the curly braces `{}` and the `return` keyword.

   ```
   const multiplyShort = (a, b) => a * b; // The result of a * b is automatically r
   eturned.
   ```

2. **Single Parameter:** If there is only one parameter, you can remove the parentheses `()`.

   ```
   const square = x => x * x;
   ```

You've already seen this in action with the array sort method: `(a, b) => a - b`. This is a concise arrow function that implicitly returns the result of `a - b`.

Part 3: Advanced Parameter Concepts

A. Default Parameters (ES6)

You can provide a default value for a parameter in case it's not passed in (i.e., it's `undefined`).

```
// The default value for 'name' is "Guest".
function greet(name = "Guest") {
  console.log(`Hello, ${name}!`);
}

greet("Alice"); // Hello, Alice!
greet();        // Hello, Guest!
```

B. Rest Parameters (`...`) (ES6)

This allows a function to accept an **indefinite number of arguments** and gather them into a **true array**.

First Thought: "Put the rest of the ingredients into a single bag."

```
// The '...numbers' gathers all arguments passed into an array called 'number
s'.
function sumAll(...numbers) {
  let total = 0;
  for (const num of numbers) {
    total += num;
  }
  return total;
}

console.log(sumAll(1, 2));          // 3
console.log(sumAll(1, 2, 3, 4, 5));   // 15
console.log(sumAll(10));            // 10
```

The rest parameter must be the **last** parameter in the function definition.

Part 4: Scope - Where Do Variables Live?

First Thought: "A function is its own little world."

- **Global Scope:** Variables declared outside of any function are "global" and can be accessed from anywhere.
- **Function Scope (Local Scope):** Variables declared inside a function (with `let`, `const`, or `var`) **only exist inside that function**. They cannot be accessed from the outside.

```
let globalVar = "I am global";

function myFunction() {
  let localVar = "I am local";
  console.log(globalVar); // "I am global" (Can access global variables)
  console.log(localVar);  // "I am local"
```

```
}

myFunction();

// console.log(localVar); // ReferenceError: localVar is not defined
```

This is a critical feature that prevents variables from different parts of your code from interfering with each other.

Rest vs Spread: The Same Syntax, Opposite Purposes

Here's the mind-bending part: **they use the exact same syntax (`...`) but do opposite things!**

The Key Difference

- **Spread** = **Unpack/Expand** (takes one thing and breaks it apart)
- **Rest** = **Pack/Collect** (takes many things and combines them into one)

Think of it like a suitcase:

- **Spread** = Opening a suitcase and spreading clothes everywhere
- **Rest** = Gathering scattered clothes and packing them into a suitcase

---

SPREAD Operator (Expanding/Unpacking)

Takes an array or object and **spreads its elements out**.

Arrays

```
const numbers = [1, 2, 3];
```

```
// Spread: Unpack the array into individual elements
console.log(...numbers);  // Output: 1 2 3 (three separate values)
console.log(numbers);     // Output: [1, 2, 3] (one array)

// Practical use: Combining arrays
const moreNumbers = [4, 5, 6];
const combined = [...numbers, ...moreNumbers];
console.log(combined);  // [1, 2, 3, 4, 5, 6]

// Practical use: Copying an array
const copy = [...numbers];

// Practical use: Passing array elements as function arguments
function add(a, b, c) {
  return a + b + c;
}
console.log(add(...numbers));  // Same as add(1, 2, 3)
```

Objects

```
const person = { name: "Alice", age: 25 };

// Spread: Unpack object properties
const updatedPerson = { ...person, city: "NYC" };
console.log(updatedPerson);
// { name: "Alice", age: 25, city: "NYC" }

// Overwriting properties
const olderPerson = { ...person, age: 30 };
// { name: "Alice", age: 30 }
```

REST Operator (Collecting/Packing)

Takes **multiple individual elements** and **collects them into an array**.

In Function Parameters

```
// Rest: Collect all remaining arguments into an array
function sum(...numbers) {
    console.log(numbers);  // numbers is an array
    return numbers.reduce((total, num) => total + num, 0);
}

console.log(sum(1, 2, 3, 4, 5));  // 15
// Inside the function, numbers = [1, 2, 3, 4, 5]

// Mixing regular params with rest
function introduce(greeting, ...names) {
    console.log(greeting);  // "Hello"
    console.log(names);     // ["Alice", "Bob", "Charlie"]
}

introduce("Hello", "Alice", "Bob", "Charlie");
```

In Destructuring (Arrays)

```
const numbers = [1, 2, 3, 4, 5];

// Rest: Collect the "rest" of the elements
const [first, second, ...others] = numbers;

console.log(first);   // 1
console.log(second);  // 2
console.log(others);  // [3, 4, 5] - collected into an array!
```

In Destructuring (Objects)

```
const person = {
    name: "Alice",
    age: 25,
```

```
    city: "NYC",
    country: "USA"
};

// Rest: Collect remaining properties
const { name, ...otherInfo } = person;

console.log(name);      // "Alice"
console.log(otherInfo);  // { age: 25, city: "NYC", country: "USA" }
```

Side-by-Side Comparison

```
// SPREAD: Taking an array and spreading it out
const arr = [1, 2, 3];
console.log(...arr);  // 1 2 3 (three separate values)

// REST: Taking separate values and collecting them
function example(...params) {
    console.log(params);  // [1, 2, 3] (one array)
}
example(1, 2, 3);
```

How to Remember Which is Which

Look at WHERE it's used:

1. **RIGHT side of assignment** = SPREAD (expanding)

   ```
   const copy = [...original];  // Spreading original
   ```

2. **LEFT side of assignment** = REST (collecting)

   ```
   const [first, ...rest] = array;  // Collecting rest
   ```

3. **Function call** = SPREAD (unpacking arguments)

```
myFunction(...array);  // Spreading array into arguments
```

4. **Function definition** = REST (collecting parameters)

```
function myFunction(...params) { }  // Collecting params
```

Real-World Example: Both Together!

```
function logDetails(name, age, ...hobbies) {
    // REST: hobbies collects remaining arguments into array
    console.log(`${name} is ${age} years old`);
    console.log("Hobbies:", hobbies);
}

const person = ["Alice", 25, "reading", "coding", "hiking"];

// SPREAD: Unpack array into individual arguments
logDetails(...person);

// Output:
// Alice is 25 years old
// Hobbies: ["reading", "coding", "hiking"]
```

Quick Rule

- **Spread** = `...` takes ONE thing → makes it MANY
- **Rest** = `...` takes MANY things → makes it ONE

Arrow Function In-depth

In-Depth Guide: Arrow Functions (`=>`)

1. The Core Purpose: A Better `function`

First Thought: Arrow functions were created to solve two major problems with traditional `function` expressions:

1. They were too verbose for simple, one-line operations (like in array methods).
2. Their behavior with the `this` keyword was a constant source of confusion and bugs.

An arrow function is a **syntactically compact alternative to a regular function expression**.

---

2. Syntax Breakdown: From Verbose to Concise

Let's transform a traditional function expression into its shortest possible arrow function form.

Start: Traditional Function Expression

```
const add = function(a, b) {
  return a + b;
};
```

Step 1: Replace `function` with `⇒`
Remove the `function` keyword and place a "fat arrow" `⇒` after the parameters.

```
const add = (a, b) ⇒ {
  return a + b;
};
```

Step 2: Implicit Return
This is the biggest syntax win. If the function body consists of only **a single `return` expression**, you can remove the curly braces `{}` and the `return` keyword. The result of the expression is returned automatically.

```
const add = (a, b) => a + b;
```

Step 3: Parentheses for a Single Parameter
If the function has **exactly one parameter**, you can also remove the parentheses around it.

```
// Traditional
const square = function(x) {
  return x * x;
};

// Arrow function with one parameter
const squareArrow = x => x * x;
```

If you have zero or more than one parameter, the parentheses are **required**.

```
const sayHello = () => "Hello"; // Zero parameters
const sum = (a, b, c) => a + b + c; // Three parameters
```

Important Gotcha: Returning an Object Literal
If you want to implicitly return an object literal, you must wrap it in parentheses to distinguish it from a function body's curly braces.

```
// WRONG: This is interpreted as a function body with a label, returning undefi
ned.
const createUser = name => { name: name };

// CORRECT: Wrap the object in parentheses.
const createUserCorrect = name => ({ name: name });
```

Callback Function

A callback function in JavaScript is a function passed as an argument to another function. This allows the receiving function, often called a higher-order function, to execute the callback function at a specific point during its operation, typically after a particular task is completed.

That's the entire concept.

The Analogy: Ordering a Pizza

Let's break this down.

The Scenario: You want a pizza, but it's going to take a while to cook. You don't want to just stand at the counter and wait. You want to go do something else.

1. **You place your order (You call a function).**
   You go to the pizza place and say, "I'd like a large pepperoni pizza." This is you calling a function, `orderPizza()`.
2. **You provide the callback (You give them your phone number).**
   The cashier says, "Great, that will be 20 minutes. What's your number so we can call you when it's ready?"
   That phone number is your **callback function**. It's the instruction you are leaving behind for what to do *after* the main task (making the pizza) is complete.

   Your "phone number" function might be called `pickUpPizza`.
3. **The long task happens (The pizza is made).**
   You leave and go about your business. The pizza place is busy making your pizza. This is the asynchronous part—a task is happening in the background, and your main program isn't blocked waiting for it.
4. **The task finishes, and they "call you back."**
   After 20 minutes, the pizza is ready. The cashier looks at your order, finds your phone number (`pickUpPizza`), and calls it.
   This is the **callback execution**.

The `orderPizza` function is now "calling back" the `pickUpPizza` function that you provided earlier.

The Code Equivalent

Let's translate this directly into JavaScript code.

```
// This is your callback function. It's the "phone number" you'll leave behind.
// It describes what to do once the pizza is ready.
function pickUpPizza() {
  console.log("Pizza is ready! Driving to the store to pick it up.");
}


function orderPizza(callback) {


  console.log("Placing the pizza order...");

    console.log("Pizza is cooked!");

    callback();


}

// --- Let's run the program ---
// We call orderPizza and give it our pickUpPizza function as the callback argu
ment.
orderPizza(pickUpPizza);

// This line will run immediately, while the pizza is still "cooking".
console.log("I'm not waiting at the store. I'm at home, coding.");
```

Output of the Code:

Placing the pizza order...
I'm not waiting at the store. I'm at home, coding.
(after 2 seconds)
Pizza is cooked!
Pizza is ready! Driving to the store to pick it up.

Scope and Closures: The Complete Deep Dive

Part 1: SCOPE - What Can See What?

What is Scope?

Scope = The **visibility** and **accessibility** of variables. It answers: "From where can I access this variable?"

Think of scope like **rooms in a house**:

- Variables declared in a room are accessible in that room
- You can look OUT from inner rooms to outer rooms
- You CANNOT look IN from outer rooms to inner rooms

The Three Types of Scope

1. Global Scope (The House Itself)

Variables declared outside any function or block are **global**.

```
const globalVar = "I'm global";

function someFunction() {
  console.log(globalVar);  // ✅ Can access
}

someFunction();  // "I'm global"
console.log(globalVar);  // ✅ Can access from anywhere
```

Real-world analogy: Like the air outside - everyone can access it.

---

2. Function Scope (A Room)

Variables declared inside a function are **only accessible inside that function**.

```
function myFunction() {
  const functionVar = "I'm in the function";
  console.log(functionVar);  // ✅ Works
}

myFunction();  // "I'm in the function"
console.log(functionVar);  // ❌ ReferenceError: functionVar is not defined
```

Key point: `var`, `let`, and `const` are all function-scoped (but `let`/`const` are also block-scoped).

---

3. Block Scope (A Closet in a Room)

Variables declared with `let` or `const` inside `{}` are **block-scoped**.

```
if (true) {
  let blockVar = "I'm in a block";
  const alsoBlockVar = "Me too";
  var notBlockScoped = "I'm different!";

  console.log(blockVar);  // ✅ Works
```

```
}

console.log(blockVar);  // ❌ ReferenceError
console.log(alsoBlockVar);  // ❌ ReferenceError
console.log(notBlockScoped);  // ✅ Works! (var ignores block scope)
```

Important: `var` is NOT block-scoped, only function-scoped!

```
function testVar() {
    if (true) {
        var x = 10;
    }
    console.log(x);  // ✅ 10 - var leaks out of block!
}

function testLet() {
    if (true) {
        let y = 10;
    }
    console.log(y);  // ❌ ReferenceError - let respects block scope
}
```

Lexical Scope (The Key Concept!)

Lexical scope = Scope is determined by **where you write the code**, not where you call it.

```
const name = "Global";

function outer() {
    const name = "Outer";

    function inner() {
        const name = "Inner";
        console.log(name);  // Which "name" will this print?
```

```
    }

    inner();
}

outer();  // "Inner"
```

Why "Inner"? JavaScript looks for variables in this order:

1. **Current scope** (inner function) → Found `name = "Inner"` ✓
2. If not found, check **outer scope** (outer function)
3. If not found, check **global scope**
4. If still not found → ReferenceError

This is called the **Scope Chain**.

The Scope Chain in Action

```
const level1 = "Global";

function outer() {
    const level2 = "Outer";

    function middle() {
        const level3 = "Middle";

        function inner() {
            const level4 = "Inner";

            // Can access ALL outer scopes!
            console.log(level4);  // "Inner"
            console.log(level3);  // "Middle"
            console.log(level2);  // "Outer"
            console.log(level1);  // "Global"
        }
```

```
    inner();
  }

  middle();
}

outer();
```

Visual representation of scope chain:

```
inner() scope
  ↓ (can look up)
middle() scope
  ↓ (can look up)
outer() scope
  ↓ (can look up)
Global scope
```

But you CANNOT look down:

```
function outer() {
  function inner() {
    const secret = "Hidden";
  }

  inner();
  console.log(secret);  // ❌ ReferenceError - can't look INTO inner function
}
```

Part 2: CLOSURES - Functions Remember Their Birthplace

What is a Closure?

Closure = A function that **remembers** variables from its **outer scope** even after the outer function has finished executing.

This is THE most important concept for understanding JavaScript!

The Basic Closure Example

```
function outer() {
    const message = "Hello";

    function inner() {
        console.log(message);  // Accesses outer's variable
    }

    return inner;  // Return the function
}

const myFunction = outer();  // outer() finishes executing
myFunction();  // "Hello" - but how does it still remember "message"?
```

What just happened?

1. `outer()` runs and creates `message`
2. `inner()` is defined INSIDE `outer()` - it "closes over" `message`
3. `outer()` returns `inner` and finishes
4. Normally, `message` would be garbage collected... **BUT**
5. `inner` still has a reference to `message` - this is a **closure**!
6. When we call `myFunction()` (which is `inner`), it still remembers `message`

Why Closures Exist

Without closures: Functions would only access their own variables and globals.

With closures: Functions can "carry" their environment with them!

```
function createCounter() {
  let count = 0;  // Private variable

  return function() {
    count++;  // Accesses outer variable
    return count;
  };
}

const counter = createCounter();

console.log(counter());  // 1
console.log(counter());  // 2
console.log(counter());  // 3

// "count" is NEVER directly accessible!
console.log(count);  // ❌ ReferenceError
```

Key insight: `count` lives on even though `createCounter()` finished! The returned function "closes over" it.

Real-World Example 1: Private Variables

Closures let you create **truly private** variables!

```
function createBankAccount(initialBalance) {
  let balance = initialBalance;  // PRIVATE - can't be accessed directly

  return {
    deposit: function(amount) {
      balance += amount;
      return balance;
    },

    withdraw: function(amount) {
```

```
            if (amount > balance) {
                return "Insufficient funds";
            }
            balance -= amount;
            return balance;
        },

        getBalance: function() {
            return balance;
        }
    };
}

const myAccount = createBankAccount(100);

console.log(myAccount.getBalance());  // 100
myAccount.deposit(50);  // 150
myAccount.withdraw(30);  // 120

// Can't directly access or modify balance!
console.log(myAccount.balance);  // undefined
myAccount.balance = 9999999;  // Doesn't work!
console.log(myAccount.getBalance());  // 120 - still protected
```

Why this works: All three methods (`deposit`, `withdraw`, `getBalance`) are closures that remember the `balance` variable!